



A Long Short-Term Memory for AI Applications in Spike-based Neuromorphic Hardware

Arjun Rao^{1,4}, Philipp Plank^{1,2,4}, Andreas Wild³ and Wolfgang Maass¹✉

Spike-based neuromorphic hardware holds promise for more energy-efficient implementations of deep neural networks (DNNs) than standard hardware such as GPUs. But this requires us to understand how DNNs can be emulated in an event-based sparse firing regime, as otherwise the energy advantage is lost. In particular, DNNs that solve sequence processing tasks typically employ long short-term memory units that are hard to emulate with few spikes. We show that a facet of many biological neurons, slow after-hyperpolarizing currents after each spike, provides an efficient solution. After-hyperpolarizing currents can easily be implemented in neuromorphic hardware that supports multi-compartment neuron models, such as Intel's Loihi chip. Filter approximation theory explains why after-hyperpolarizing neurons can emulate the function of long short-term memory units. This yields a highly energy-efficient approach to time-series classification. Furthermore, it provides the basis for an energy-efficient implementation of an important class of large DNNs that extract relations between words and sentences in order to answer questions about the text.

Energy consumption is a major impediment to more widespread applications of new AI methods that use deep neural networks (DNNs), especially in edge devices. Spike-based neuromorphic hardware is one direction that promises to alleviate this problem. This research is partially inspired by the brain, which runs even more complex and larger neural networks with a total energy consumption of just 20 W. A key factor of this astounding energy efficiency is that neurons in the brain emit a signal (spike) on average just a few times per second. In contrast, the units of a typical DNN emit an output value, and hence consume energy, several orders of magnitudes more frequently. But determining which types of DNN can be implemented in an energy-efficient manner with sparsely active neurons in neuromorphic hardware for modern AI solutions remains an open problem (see fig. 9 of Davies et al.¹). In most cases this requires a rethinking of DNN design principles.

A more specific open problem is how the long short-term memory (LSTM) units of DNNs for sequence processing tasks can be implemented in spike-based neuromorphic hardware with good energy efficiency. We show that a characteristic feature of biological neurons, the presence of slowly changing internal currents that have so far not been included in neuromorphic hardware models, endow spiking neural networks (SNNs) with similar working memory capabilities to LSTM units in DNNs. In particular, slow after-hyperpolarizing (AHP) currents reduce the readiness of biological neurons to fire again after recent firing activity. This effect is called spike frequency adaptation in neuroscience (see Benda et al.² and Gutkin et al.³). Experimental data from the Allen Institute⁴ show that a considerable number of neurons in the neocortex (over 30% of excitatory neurons in the human frontal lobe) exhibit spike frequency adaptation. We show that AHP neurons not only save energy by reducing firing activity, but also provide a genuine alternative to LSTM units for solving sequence processing tasks, and support training through backpropagation through time (BPTT).

Another major difference between biological neurons and standard spiking neuron models is that biological neurons keep their membrane potential in a relatively narrow regime. In contrast, the

membrane potential of models often assumes extremely negative values when the network is trained with regularization terms to induce low firing rates. This practically removes many of them from the current network computation. We introduce a membrane voltage regularization principle that alleviates this problem, and supports the design of extremely sparsely firing spiking DNNs.

We analyse the functional implications of these two principles on a commonly used spike-based chip: Intel's neuromorphic chip Loihi⁵, and find a notable reduction in the energy-delay product (EDP). In contrast to power, the EDP accounts for the true energy and time cost per task/workload/computation. Simultaneously, these implementations demonstrate that two hallmarks of cognitive computations in both brains and in machine intelligence—working memory and reasoning about relations between concepts or objects—can in fact be implemented more efficiently in spike-based neuromorphic hardware than in GPUs, the standard computing hardware for implementing DNNs.

Long short-term memory for spike-based neuromorphic hardware

The value of a continuous variable can be stored in the memory cell of an LSTM unit, to which read and write access are controlled by auxiliary neurons with a sigmoidal activation function. A straightforward implementation of an LSTM unit⁶ with spiking neurons arrives at rate coding of the stored variable and the continuous-value outputs of the auxiliary gates, and hence cannot reach an energy-efficient sparse firing regime. Shrestha et al.⁷ showed that if one adds some clever subcircuits and a store-and-release mechanism to the already complex circuitry of an LSTM unit, one can emulate it on IBM's TrueNorth chip⁸, using substantially less energy than a GPU, but with larger latency. They tested computational performance on sequences of up to length 36 (compare this with a sequence length of 784 in the standard version of the sequential Modified National Institute of Standards and Technology (sMNIST) task that will be discussed in the next section). More recently, Rezaabad et al.⁹ considered a spike-based implementation of LSTM units where each

¹Institute of Theoretical Computer Science, Graz University of Technology, Graz, Austria. ²Intel Labs, Intel Corporation, Neubiberg, Germany. ³Intel Labs, Intel Corporation, Hillsboro, OR, USA. ⁴These authors contributed equally: Arjun Rao and Philipp Plank. ✉e-mail: maass@igi.tugraz.at

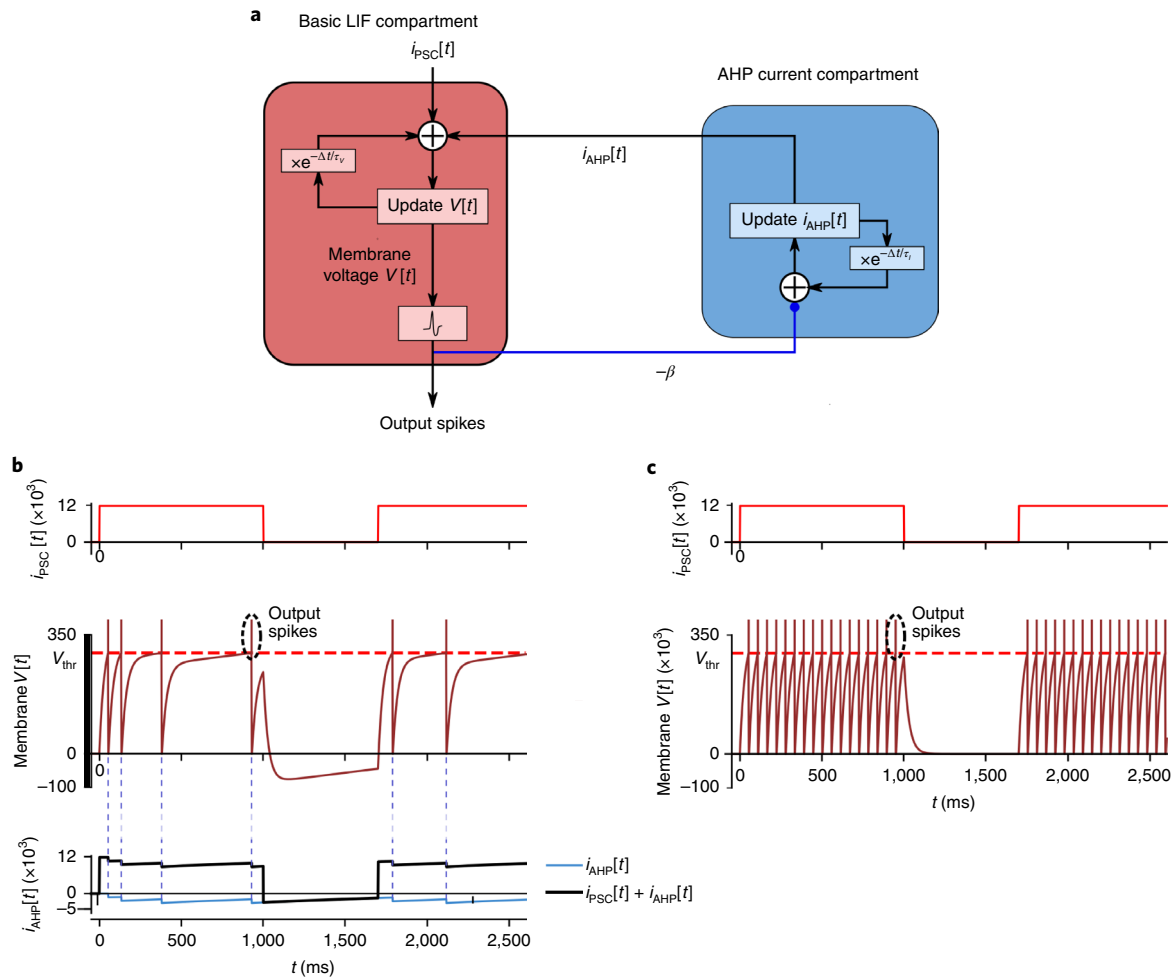


Fig. 1 | Schematics and dynamics of a two-compartment LIF neuron model with AHP currents. **a**, Schema of the two-compartment neuron model. The second compartment adds a very slowly decaying AHP current for each spike of the neuron. The AHP current (i_{AHP}) becomes more negative by an amount β whenever the neuron spikes (that is, when $z(t)=1$) and decays exponentially between spikes with a large τ_{AHP} . This current, along with the input post synaptic current i_{PSC} , are integrated to calculate the membrane voltage. This can be implemented easily on Loihi. The $i_{\text{PSC}}[t]$ caused by external input to the neuron and the AHP current are leaky integrated into the membrane voltage $V[t]$. Spikes are emitted and $V[t]$ is reset each time the voltage crosses a firing threshold V_{thr} . **b**, Response of an AHP neuron to a sequence of two current steps as input. One can see how the AHP currents (blue trace) add up and reduce firing. Note the difference in the responses of the AHP neuron to the two input current steps. It shows a clear memory effect. **c**, Response of the standard LIF neuron model to the same input currents. Here the second compartment for AHP currents in **a** has been removed. One can see that this neuron fires at a much higher constant spike rate, and responds to the second current step in exactly the same way it responds to the first one. All voltage and current values are as seen in the Loihi registers.

output of their sigmoidal gates was rounded to ‘spike’ or ‘no-spike’. They applied the resulting SNN to a simpler version of sMNIST in which each handwritten digit was transformed into a time series of length 28 (presenting a whole row of pixel values at each time step). No implementation in neuromorphic hardware, and hence no energy consumption analysis, was presented for this approach.

Brains seem to use a different method to LSTM-like units for solving temporal computing tasks. We propose that brains instead use slower dynamic processes of neurons, such as AHP currents (Fig. 1a,b). This mechanism does not enable the storage and maintenance of a continuous-value variable like in the memory cell of an LSTM unit, but it supports an alternative strategy for computing with short-term memory that is motivated by a theoretical result on filter approximations, based on the Stone–Weierstrass theorem from mathematics (see theorem 1 in Maass et al.¹⁰ for a suitable formulation). This theory implies that to classify time series, one does not have to store their earlier segments in memory cells. Instead, it suffices to satisfy the pointwise separation property (PSPR). We will

explain the PSPR in the next section in the context of an application to a concrete time-series classification task, and show that neurons with AHP currents significantly improve the PSPR.

AHP currents can be realized easily by a two-compartment version of the standard leaky integrate-and-fire (LIF) spiking neuron model (Fig. 1a), and hence by any neuromorphic hardware that supports multi-compartment neurons, such as Loihi⁵ or SpiNNaker¹¹. When the neuron fires a few spikes within a short time window, the resulting AHP currents add up (Fig. 1b) and delay or halt further firing. Their impact on firing outlasts the gap of 700 ms between the two input current steps if the time constant τ_{AHP} is sufficiently large, and reduces the firing response at the beginning of the second input current step. In this way information can be stored in the hidden variable $i_{\text{AHP}}[t]$ for longer time spans, and can be communicated to other neurons in the network or to network readouts when stimulated by an input current.

AHP currents, being inhibitory, reduce the firing activity of a neuron: compare for example the firing response of neurons with

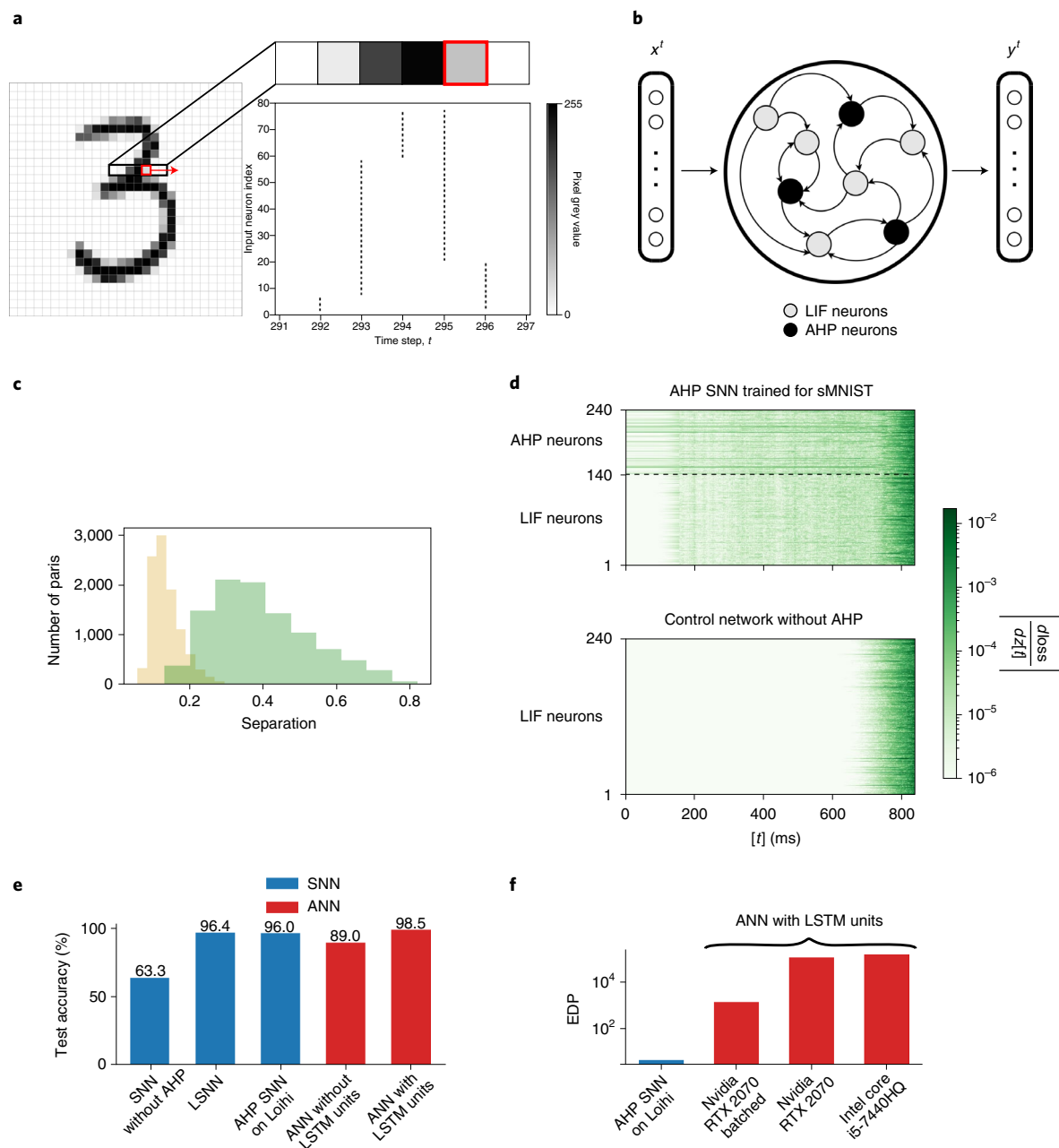


Fig. 2 | PSPR, gradient transmission and energy consumption of AHP-SNNs for the sMNIST task. **a**, The input pixels (left) are fed in sequentially, row by row, encoded by spikes on the basis of a threshold-crossing method. 40 thresholds were used, represented by 80 input neurons, which send spikes depending on the change in the pixel value with respect to the previous pixel. **b**, The AHP SNN consists of an input layer sending spikes (left), a recurrently connected network of LIF and AHP neurons (middle) and a linear readout layer (right). **c**, PSPR for SNNs with (green) and without (yellow) AHP neurons consisting of 240 fully connected neurons with random weights. Histograms of distances of the normalized network states for pairs of inputs from MNIST classes 6 and 8 are shown. **d**, Backpropagation of the gradient of loss with respect to the neuron spike output $z[t]$ through time for a sample image as input for SNNs with (top) and without (bottom) AHP neurons. AHP currents provide highways to the past in a similar way to memory cells of LSTM units and they also improve gradient transmission to neurons without AHP currents. **e**, Classification accuracy for sMNIST of the AHP SNN network on Loihi in comparison to SNNs without AHP neurons. LSNNs and non-spiking ANNs consisting of sigmoidal neurons and LSTM units were compared with the full precision LSNN; comparison results are from Bellec et al.¹². **f**, Comparison of the EDP for the AHP SNN on Loihi and a network of LSTM units of the same size on standard Nvidia GPUs utilizing parallel evaluation of 100 samples at the same time (batched) and on a standard Intel CPU.

and without AHP currents in Fig. 1b,c. Neurons with AHP currents can also be used for many standard network computations, and thus one can view SNNs that include them as in-memory-computing architectures, since they do not require energy-consuming shuffling of information between processors and working memory units. We refer to neurons with AHP currents as AHP neurons and to neurons

without AHP currents as LIF neurons. Networks that contain AHP neurons are termed AHP SNNs.

As AHP currents change on a slow timescale, the gradients of BPTT can pass through the associated hidden variable without any loss of precision (Methods). Furthermore, the gradient can pass repeatedly through these variables when it moves backwards in time

Table 1 | Benchmarking results

	sMNIST		Relational reasoning				
	GPU	CPU	GPU				
Number of cores on Loihi	1	1	124	332	700	1,552	2,320
Number of sentences (Relational Reasoning)	-	-	2	6	10	16	20
Energy ratio	7,467×	4,774×	16.49×	11.92×	7.78×	5.32×	4.36×
Latency ratio	2.82×	5.89×	0.73×	0.56×	0.44×	0.33×	0.38×
EDP ratio	21,026×	28,134×	12.10×	6.73×	3.41×	1.73×	1.67×

All ratios are shown against Loihi. Loihi: Nahuku board (ncl-ghrd-01). CPU: Intel Core i9-7920X. RAM: 128GB. OS: Ubuntu 16.04.6 LTS. NxSDK: 0.95. Nvidia RTX 2070: Nvidia RTX 2070 Super. GPU-RAM: 8GB. CPU: Intel Core i7-9700K. RAM: 32GB. OS: Ubuntu 16.04.6 LTS. Python 3.6.5. TensorFlow-GPU: 1.14.0. CUDA: 10.0. Intel Core i5-7440HQ: RAM: 16GB. OS: Windows 10 (build18362). Python 3.6.7. TensorFlow: 1.14.1 Performance results are based on testing as of October 31, 2021 and may not reflect all publicly available security updates. Results may vary. Comparisons and scaling analysis of the spiking relational network on Loihi against the corresponding ANN on a CPU (Intel Core i5-7440HQ) and a GPU (Nvidia RTX 2070). For the scaling analysis of the spiking RelNet the dataset was grouped by the number of sentences per sample, which in turn determines the number of configured AHP SNNs and therefore neuro-cores per sample. All measurements were done using 250 input samples, except for network size 16, for which only 100 samples were used, as there were not enough test samples containing exactly 16 sentences. The energy per inference was calculated using total power values. More detailed results can be seen in the Supplementary Information.

without being subject to exponential growth or decay (see Fig. 2d for a demonstration). In this way, gradients propagate substantially better even in earlier time slices of the neurons without AHP currents, as they can use the slowly changing AHP currents of other neurons as intermediate stepping stones. Therefore AHP SNNs can, in general, be trained more effectively through BPTT than SNNs consisting of only standard LIF neurons. Long short-term memory spiking neural networks (LSNNs), introduced by Bellec et al.¹², provide similar advantages, but employ neurons with a time-varying firing threshold that cannot be efficiently implemented on Loihi.

Energy consumption for time-series classification

To test the energy efficiency of the proposed replacement of LSTM units via AHP SNNs, we used a classical time-series classification task: sMNIST. Here the 784 pixel values of each 28×28 pixel image of a handwritten digit from the MNIST dataset¹³ were presented sequentially, one pixel at each millisecond time step, in a fixed order: row by row, starting with the top row (see the left part of Fig. 2a). The grey values of pixels are encoded by spikes through a population of spiking input neurons that fire when the grey value crosses some threshold, where each neuron in the population relates to specific thresholds (see bottom right of Fig. 2a). The computational task was to classify the underlying digit from the time series of the pixel values that were presented sequentially over a period of 784 ms. The network output was encoded by whichever of the ten readout neurons had the highest membrane potential at time 840 ms; that is, 56 ms after the pixel sequence was provided. An additional input neuron fired during these 56 ms. This time-series classification task was obviously more difficult than the standard version of MNIST¹³ with batch inputs, as information from pixels in the upper pixel rows needs to be remembered and combined with information from pixels in lower rows. This difficulty is obvious for the case of digits 6 and 8, for example, where the most salient differences are contained in the earliest part of the resulting input time series.

To solve this time-series classification task, a recurrent network needs to have different network states for input sequences that represent different digits after time $T=784$ ms, when the input sequence finishes; otherwise it could not activate different readouts for these input sequences. The cited theoretical result on filter approximations states that this necessary condition is also sufficient under some constraints. In other words, a neural network should aim to satisfy the PSPR. The PSPR demands in this case that the network states at T should be different for any pair of input time series that require different network outputs, particularly for any pair of handwritten digits 6 and 8. As the readouts from the recurrent network need to work in the presence of noise, which arises

here from different writing styles of the same digit, one should try to satisfy a stronger version of the PSPR: that the network states at T are not only different, but also significantly different. We show in Fig. 2c the histogram of pairwise Euclidean distances between normalized network states (low-pass-filtered spike trains of all network neurons) at $T=840$ ms for pairs of digits 6 and 8. This is shown for two variants of the network in Fig. 2b, using in both cases random weights for recurrent connections, rather than trained weights: one containing AHP neurons, and one without AHP neurons. One can clearly see that AHP neurons substantially enhance the separation of network states. This improved separation property via AHP neurons is reflected in the time-series classification capability of these untrained networks: with AHP neurons, trained linear readouts from the considered network states at T achieve an accuracy of 89.8% for the classification of digits 6 and 8, whereas an accuracy of only 69.6% was achieved without AHP neurons.

Another important property of LSTM units is that they can be trained through BPTT without gradients exploding or vanishing when they are backpropagated in time. In Fig. 2d, we show the gradient of the loss propagating through an AHP SNN network fully trained on the sMNIST task (top) versus a randomly initialized control network without AHP neurons (bottom). We demonstrate here that the AHP currents of multi-compartment neurons provide a corresponding 'highway to the past' for gradient propagation because these currents change only slowly.

We examined solutions of this time-series classification task from various types of recurrent neural network, particularly for AHP SNNs that contained both AHP neurons and LIF neurons without AHP (Fig. 2b). Figure 2e shows the performance of an AHP SNN consisting of 240 neurons, 100 of which were AHP neurons. To arrive at a recurrent network that can be efficiently implemented on Loihi in spite of its limitation on the number of synaptic connections (Methods), we considered AHP SNNs with just 20% connectivity. To achieve high performance despite this, we used BPTT and deep rewiring (DEEP-R)¹⁴ for training, where synaptic connections were occasionally removed and randomly added to arrive at a well-functioning network architecture. This adaptive rewiring strategy was inspired by the ongoing removal of weak synaptic connections (spines) and the creation of new ones in biological neural networks. For comparison, we also show performance results for several other spiking and non-spiking neural networks for the same task. In spite of the fact that weight resolution was limited to 8 bits in the Loihi implementation of the AHP SNN, its performance was on the same high level as other spiking and non-spiking networks that were not subject to this constraint. The comparison between the performance of the SNNs with and without AHP neurons shows that the high performance level of AHP SNNs is only to a minor

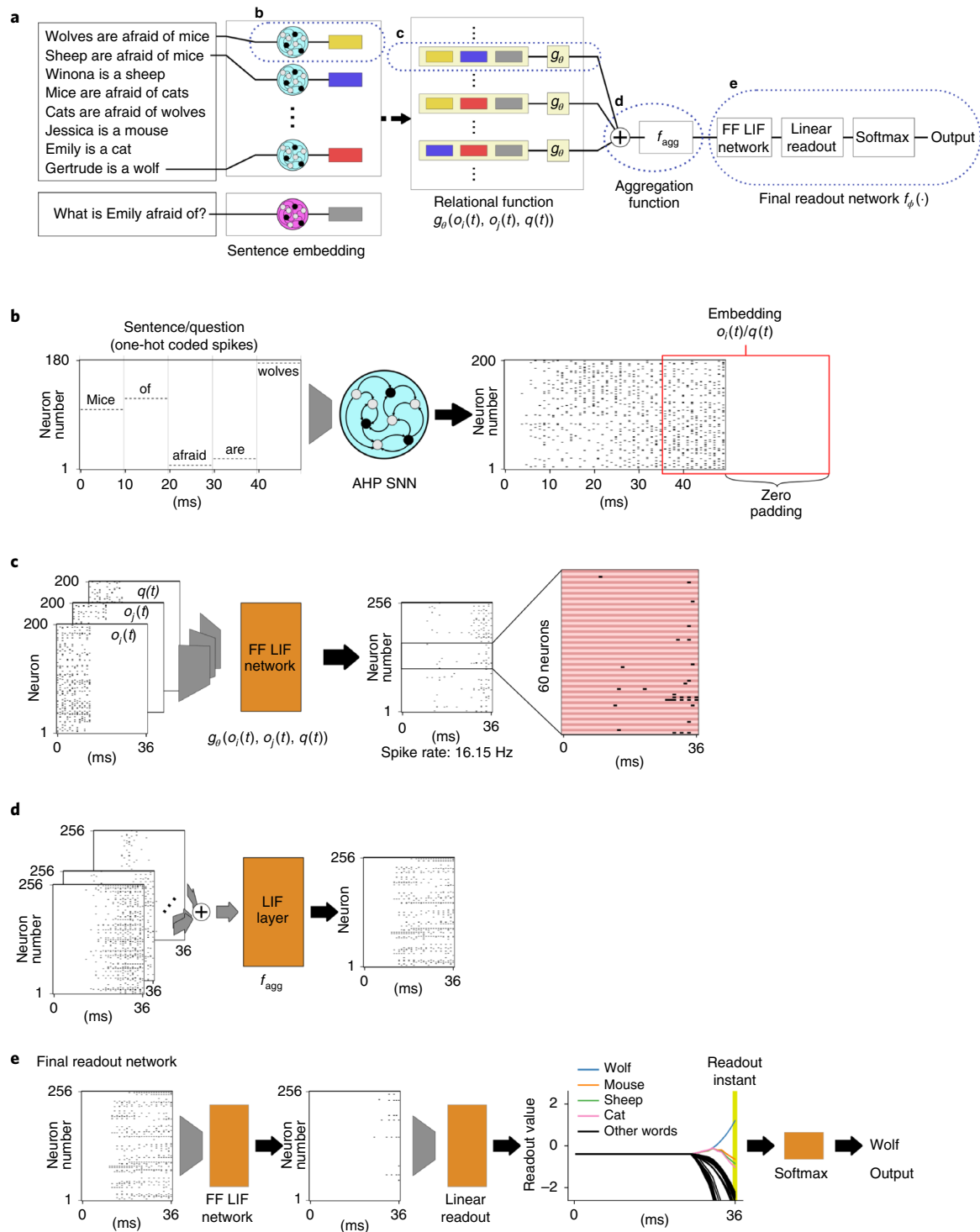


Fig. 3 | Spiking RelNet implementation with very sparse firing. **a**, The top-level spiking RelNet architecture. We embed each sentence and the question into spike sequence objects $o_i(t)$ and $q(t)$ respectively via an AHP SNN. For each pair of sentence objects $o_i, o_j: 1 \leq i \leq j \leq 20$, we apply the relational function g_θ to the triplet $(o_i(t), o_j(t), q(t))$. The outputs of g_θ are aggregated in a LIF neuron layer f_{agg} and then passed to the final readout function f_ϕ . The blue dotted ovals mark the different parts of the spiking RelNet shown in **b–e**. **b**, The embedding scheme, where each word is provided for $T_{\text{word}} = 10$ ms with one-hot coded spikes, aligned so that the first word is provided at the very end of the duration. The spikes during the last 14 ms of the sentence representation are padded with a period of 23 ms without spikes (red box) to form a time-compressed sentence embedding $o_i(t)$ and $q(t)$. **c**, An instance of the spiking g_θ operating on a sample triplet $(o_i(t), o_j(t), q(t))$. The zoomed region of the spike raster for 60 sample neurons makes the resulting very sparse firing activity throughout the network apparent (alternating light/dark shading demarcates the 60 neurons). **d**, The aggregation layer is a layer of LIF neurons that receive one-to-one connections from each g_θ instance. This aggregates the spike trains from the g_θ instances and outputs a spike sequence for the readout network. **e**, The final readout function consists of a three-layer feed-forward (FF) LIF neuron network followed by a linear readout (with one neuron per word in the dictionary) that integrates synaptic inputs only during the last 10 ms. The value of the readout at the final time step (marked by the yellow bar) provides input to the softmax, whose output produces the final answer through one-hot encoding of words.

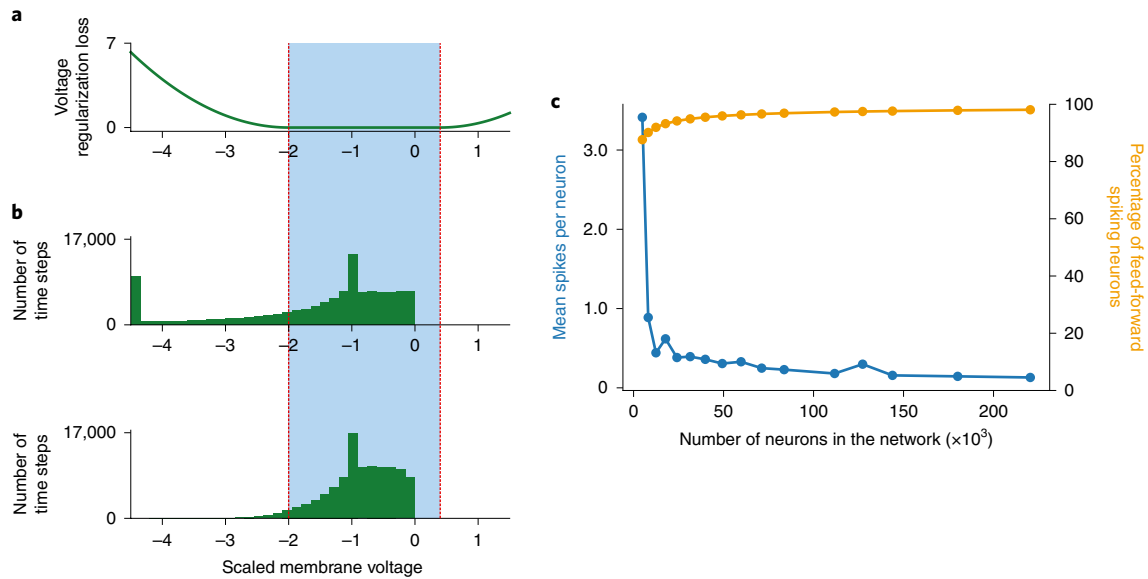


Fig. 4 | Voltage regularization and its capability to enforce a sparse firing regime in combination with spike rate regularization. a, The voltage regularization penalty as function of the value taken by the scaled membrane voltage at a particular time step. The scaled membrane voltage is defined in equation (11). A value of 0 corresponds to the spiking threshold, and a value of -1 corresponds to the voltage for a zero input PSC. The membrane voltage is thus penalized if the scaled voltage is outside the blue shaded range $[-2, 0.4]$. **b,** The distribution of the scaled voltage values across different batches, neurons and time steps with (top) and without (bottom) regularization. **c,** The spikes used per neuron and the percentage of feed-forward spiking neurons in relation to the network size (which varies for different story sizes). One can see that larger networks use fewer spikes per neuron as a result of spike rate regularization combined with voltage regularization, which results in savings in energy when run in hardware.

extent due to their common recurrent SNN architecture. Details on the architecture and parameters of all networks can be found in the Methods.

To compare accuracy, latency and energy consumption of LSTM networks in conventional hardware with that of the AHP SNN on Loihi, we measured the EDP of an LSTM network⁶ for the same task on CPUs and GPUs. The EDP of the AHP SNN running on Loihi was by four orders of magnitude lower than that of the traditional LSTM architecture on a CPU or on a GPU (Fig. 2f) for batch size 1, with Loihi outperforming by over two times on latency and over a 1,000 times on energy consumption per inference (Table 1). Details of the benchmarking procedure can be found in Supplementary Sections 2–4.

One reason for this marked improvement in latency and energy consumption compared with LSTMs in conventional hardware is the relatively small network size that is sufficient to solve this task. Thus, the network fits on a single Loihi chip and uses only one neuro-core. The neuro-core is the fundamental computational unit of Loihi that has a shared static random-access memory (SRAM). Thus keeping spike traffic within a neuro-core is the fastest and most efficient way to process spikes with Loihi. Performance in conventional neural networks is typically increased by using parallel processing of batches of data to increase the throughput. Even with a batch size of 100 on the GPU the spiking network on Loihi operating in the batch size 1 regime is still more energy efficient.

An energy-efficient spike-based DNN for relational reasoning

We wondered whether this implementation of working memory in spiking neurons could also be used to implement large DNNs for more demanding sequence processing tasks in an energy-efficient manner in spike-based neuromorphic hardware. We therefore implemented and tested a spiking variant of the relational network (RelNet) of Santoro et al.¹⁵ on Loihi, which we refer to as the spiking RelNet. The question of whether this can be done in an

energy-efficient manner is non-trivial as the spiking RelNet consists primarily of feed-forward networks. The spiking RelNet implements a function $\text{RN}([o_1(t), o_2(t), \dots, o_K(t)], q(t))$ which takes as input a set of K objects (sentences) and a single question, which are encoded respectively by input spike trains $o_i(t), \dots, o_K(t)$ and $q(t)$ (Fig. 3b). As indicated in Fig. 3a, it computes the function

$$\text{RN}([o_1(t), o_2(t), \dots, o_K(t)], q(t)) = f_\phi \left(f_{\text{agg}} \left(\sum_{1 \leq i \leq j \leq N} g_\theta(o_i(t), o_j(t), q(t)) \right) \right), \quad (1)$$

with the output given through one-hot encoding of words by read-out units. The only recurrent network modules are the ones labelled 'b' in Fig. 3a, which transform each input sequence (a sentence of words in natural language) into spiking activity of 200 neurons within a compressed time span of 37 ms. This input embedding of sentences was carried out by LSTM networks in the RelNet¹⁵, and is carried out by AHP SNNs in the spiking RelNet. In the next processing step (Fig. 3c), the resulting compressed spike codes for each pair of sentences in the story and for the question are processed in parallel by a copy of a feed-forward LIF neuron network that implements the relational function $g_\theta(o_i(t), o_j(t), q(t))$ that extracts salient relational information for q from the two sentences. The outputs of these network modules are superimposed and connected to a LIF neuron layer one-to-one, which implements the element-wise function f_{agg} (Fig. 3d). The readout network f_ϕ processes the output of f_{agg} through another feed-forward LIF neuron network. The feed-forward networks do not use the AHP current. The answer to q is then given by an application of softmax to one-hot readout neurons that each favour a particular word as the answer (Fig. 3e). For more details, see the Methods and Supplementary Section 4.

A large fraction of the spiking RelNet is formed by feed-forward layers, especially as the number of sentences increases (Fig. 4b). This is problematic from the perspective of energy efficiency, as

previous emulations of feed-forward networks in spike-based hardware suggested that their potential energy and latency advantages are hampered by the need for relatively large firing rates, instead of event-based processing, to achieve good performance¹. In contrast, our spiking RelNet uses, on average, substantially less than one spike per neuron for the whole computation, see Fig. 4c. We achieved this through an aggressive spike rate regularization. But as this tends to move the membrane potential of many neurons into a strongly negative regime where they can no longer participate in the network computation, we complemented that by a new membrane voltage regularization that keeps membrane voltages in a suitable range. We also moved the DNN into an event-based processing regime by forcing it to produce its network output at a particular time point, and by using neurons without a refractory period and a relatively short membrane time constant of 7 ms (see Supplementary Table 3 for data on the impact of these parameters).

The hardware implementation required changes to the network architecture including restricting the ordered pairs to $1 \leq i \leq j \leq N$ to allow the network to efficiently infer sentence ordering, as well as the introduction of f_{agg} to tackle fan-in constraints. The network that solved the longest stories (20 sentences) contained 238,604 neurons. The optimal implementation of this network on Loihi, conforming to connectivity constraints (see the Methods and Supplementary Section 4.5), required 2,308 neuro-cores spread across 22 chips (128 neuro-cores per chip; Fig. 5b,c), which increased the energy consumption compared with the sMNIST case. We also demonstrate a method of placing the feed-forward layers across Loihi chips that minimizes congestion during spike transport (Fig. 5d), which provided substantial improvements in the EDP (Fig. 5e; see details in the Methods and Supplementary Information). As the spiking RelNet requires much less than 1 spike per neuron, it is likely to provide substantially larger energy savings in future hardware that does not have this connectivity constraint.

We tested the performance of the resulting spike-based RelNet implementation on Loihi for a standard benchmark dataset for question-answering: the bAbI dataset introduced by Weston et al.¹⁶, which was also used for testing RelNet by Santoro et al.¹⁵. This dataset consists of 20 different types of task that each probe different challenges in reasoning about relational information contained in a set of sentences (that is, a story). The questions are formulated in such a way that an answer can be given with a single word via one-hot encoding in the output (or with a sequence of two words in the case of path planning in Task 19; one has here an output category for each such possible sequence). According to the convention of Weston et al.¹⁶ and Santoro et al.¹⁵, a task is considered to be solved if the network has an error rate of at most 5% on instances of the task that had not been used for training. When applying a RelNet to solve this task, each sentence (question) forms an object $o_i(q)$ that is embedded via AHP SNNs to a spiking representation $o_i(t)(q(t))$, which is then processed as described above. The details of input encoding are discussed in the Methods.

We trained the spiking RelNet on the combined data of 17 out of 20 bAbI tasks, excluding those that the non-spiking RelNet was not able to solve. We observed that the spiking RelNet could solve 16 out of the 17 tasks with an error rate under 5%. We show in

Supplementary Table 3 that networks using more time steps and larger time constants, with consequently greater energy consumption and latency, solved all 17 tasks.

Energy efficiency of RelNet in spike-based neuromorphic hardware. We compared the energy consumption and delay of the spike-based implementation of RelNet on Loihi with GPU implementations of the ANN RelNet from Santoro et al.¹⁵ (Table 1). It can be seen that the spike-based implementation consumes between 4 and 16 times less energy than the GPU implementation. The energy savings are lower for longer story sizes, seemingly because these require the use of substantially more Loihi chips and interchip communication seems to be less energy efficient in this spike-based hardware. One should also note that the average length of a story for the 16 datasets that we considered is just 6.5 sentences. The computation time on Loihi was slightly larger than on the GPU. But nevertheless, the resulting EDP remained lower for Loihi. For the longest, and therefore slowest, story size the average computation time per sample was 6.54 ms wall-clock time, which would still be sufficient for online applications such as voice control or virtual assistants.

Discussion

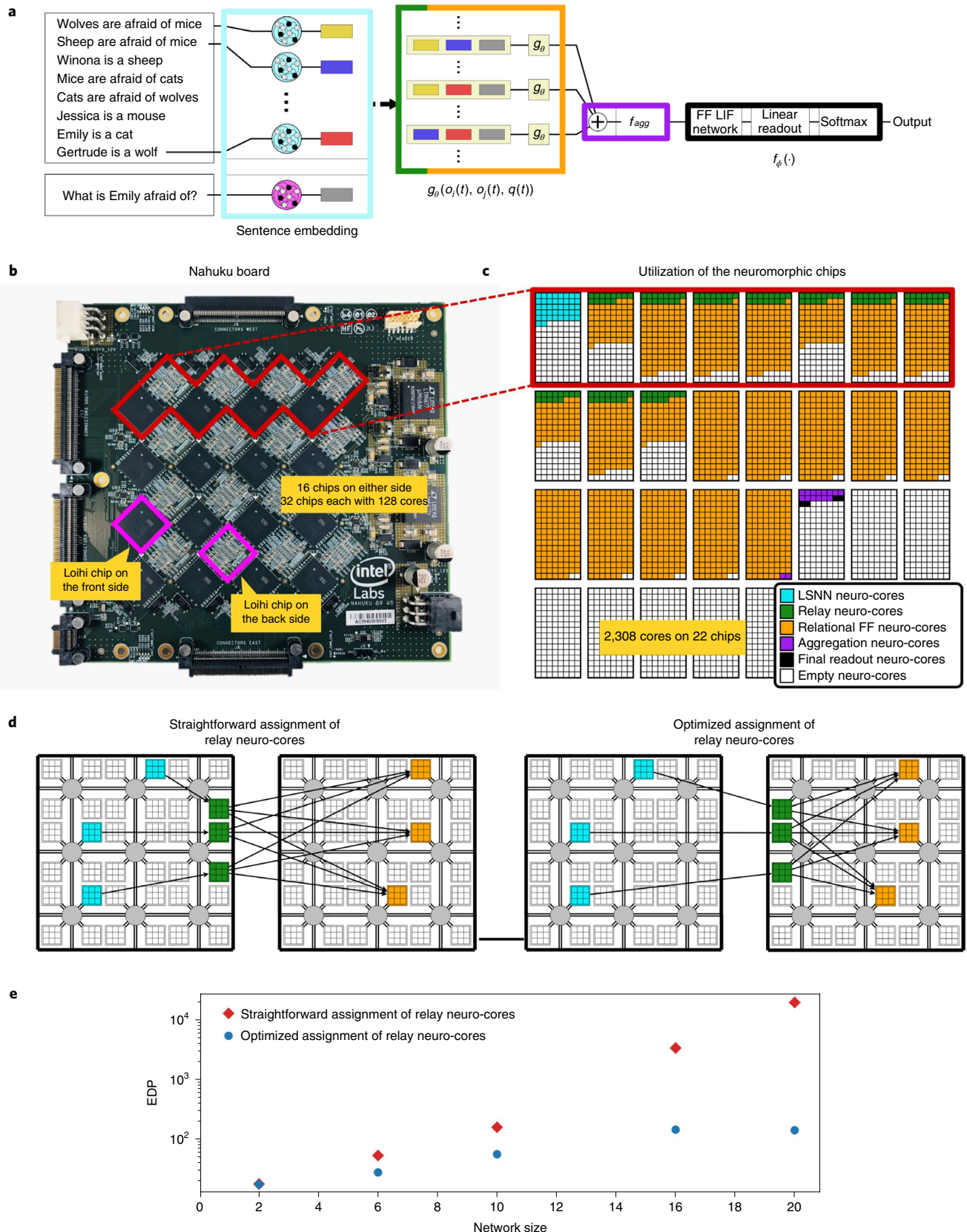
We have shown that a key tool for sequence processing by DNNs in machine learning and AI, LSTM units, can be replaced in spike-based neuromorphic hardware by neurons with a biologically inspired mechanism for spike frequency adaptation: AHP currents. This approach is supported by a theoretical principle, the PSPR from filter approximation theory (Fig. 2c). Adding a compartment for AHP currents to the neuron model also has the advantage of enhancing the results of BPTT training by creating highways to the past for the backward propagation of gradients (Fig. 2d). As AHP neurons can also be used for generic network computations, this solution does not require a separation of units for computing and working memory—it is an in-memory computing solution. This reduces the latency and energy consumption that generally arise from traffic between computing and memory units. The resulting spike-based solution for solving the benchmark time-series classification task sMNIST is three orders of magnitude more energy efficient than state-of-the-art implementations of LSTM networks on CPUs and GPUs, while achieving lower latency and virtually the same accuracy.

We have also shown that the use of AHP neurons enabled us to port large DNNs that involve large feed-forward network components, as well as LSTM units, into spike-based hardware. We have focused on the example of relational networks, as these enable a qualitative jump in AI capabilities by supporting reasoning about relationships between objects in a story or image. An energy-efficient spike-based implementation of relational networks requires methods that enable them to use rare events (spikes) in time instead of rate-based neural codes, including in their feed-forward network modules. We have shown that a sparseness of much less than one spike per neuron during the whole computation is achievable for relational networks (Fig. 4c). This became possible through a new voltage regularization method used during training, in combination with an output convention that forced the

Fig. 5 | Spiking RelNet placement and optimization on Loihi. **a**, The different parts of the spiking RelNet highlighted with the colour coding used in **c** and **d**. **b**, The Spiking RelNet was configured on a Nahuku board with 32 Loihi chips. On each side of the board 16 chips were placed in a checkerboard pattern. Each chip had 128 neuro-cores. **c**, The full-scale spiking RelNet that can solve tasks up to 20 sentences long utilizes 2,308 neuro-cores on 22 chips. The detailed mapping of the different layers is shown. To minimize cross-chip spikes, some chips were not fully utilized. **d**, Straightforward assignment of relay neuro-cores on the same chip as the source neuro-cores (left) and optimized assignment of the relay neuro-cores on the chip of the target neuro-cores, to minimize interchip spike traffic (right). Note that the target neuro-cores must be assigned carefully to do this efficiently. **e**, The benefit of the optimized assignment in terms of the EDP as a function of network size, as measured by number of bAbI sentences, which roughly corresponds to the number of Loihi chips for the RelNet solving bAbI tasks on Loihi. Thus we see that the placement of the Spiking RelNet is optimized in terms of resource utilization, as well as network performance in the hardware. Photograph in **b** © Intel Corporation.

network to produce its decision at a specific point in time, and a spiking neuron model with a short membrane time constant and no refractory period that supports computational operations that

are strictly local in time. The resulting implementation of relational networks on Loihi provides an example of a large DNN on Loihi, where this spike-based hardware becomes more energy efficient



than GPUs (see fig. 3 of Davies et al.¹). Furthermore, the resulting very sparsely active spiking RelNet is likely to become substantially more energy efficient in current and future neuromorphic hardware in which the number of synaptic connections of the neurons in a neuro-core is less constrained than on Loihi. This constraint forced us to distribute the spiking RelNet over 2,308 neuro-cores on 22 Loihi chips, thereby increasing latency and energy consumption. Despite that, a notable reduction in the EDP for relational networks compared with GPUs was demonstrated. In fact, a review of DNN implementations on Loihi¹ arrives at the conclusion that it represents “the largest deep learning network to date showing gains compared to conventional architectures”. Thus we have shown that relational networks represent a class of DNNs that can be ported more efficiently to spike-based hardware than CNNs. On the basis of the results of Santoro et al.¹⁵, one can expect that relational networks in neuromorphic hardware could be used not only to solve question-answering tasks in natural language, but also for reasoning about relations between objects in an image or in an auditory scene. This can provide a qualitative jump in the AI capabilities of energy-efficient neuromorphic hardware.

Another interesting next step would be to enable on-chip training of these spike-based alternatives to LSTM networks by using e-prop instead of BPTT, which has already been shown to work very well for LSNNs¹⁷. One-shot learning capability has also been demonstrated for these spiking networks¹⁸, and it is likely that the required method will also enable one-shot on-chip training of these networks.

Finally, adding AHP currents to spiking neuron models in neuromorphic hardware can be seen as a first step towards porting more sophisticated point neuron models for neurons in the neocortex¹⁹ into such hardware. If one adds one more current in an additional neuron compartment it will become possible to implement the diverse array of GLIF3 neuron models of arguably the most advanced model for generic cortical microcircuits from ref.¹⁹ in neuromorphic hardware. This will open the door to new uses of neuromorphic hardware in computational neuroscience: for simulating large state-of-the-art models for neural networks of the brain substantially faster and with less energy than is possible at present. This has the potential to become a major new application of neuromorphic hardware such as Loihi or SpiNNaker¹¹, supporting the implementation of such biological refinements of the standard spiking neuron model. In particular, neuromorphic hardware may become important to answering an important open question from computational neuroscience: what is the computational benefit of the astounding diversity of neuron types in the neocortex that have emerged during the evolution of the brain?

Methods

Neuron model with AHP currents. The dynamical behaviour of an LIF neuron and the AHP neuron that has AHP currents (indexed by j), as implemented on Loihi, is given by equations (3)–(6). Here we show the dynamic interaction between incoming spikes at time t , the resultant input PSC $i_{PSC,j}[t]$, the internal AHP current $i_{AHP,j}[t]$, the membrane voltage $V_j[t]$ and the output spikes $z_j[t+1]$.

$$i_{PSC,j}[t + \Delta t] = \alpha_I i_{PSC,j}[t] + \sum_i w_{ij} z_i[t - d_{ij}], \quad (2)$$

$$i_{AHP,j}[t + \Delta t] = \alpha_{AHP} i_{AHP,j}[t] - \beta z_j[t], \quad (3)$$

$$V_j[t + \Delta t] = \begin{cases} \alpha_V V_j[t] + \frac{1}{g_V} (i_{PSC,j}[t + \Delta t] + i_{AHP,j}[t + \Delta t]) & \text{if neuron is not refractory} \\ 0 & \text{otherwise} \end{cases}, \quad (4)$$

$$z_j[t + \Delta t] = \begin{cases} 1 & \text{if } V_j[t + \Delta t] > b_0 \\ 0 & \text{otherwise} \end{cases}, \quad (5)$$

$$V_j[t + \Delta t] \rightarrow 0 \text{ if } z_j[t + \Delta t] = 1. \quad (6)$$

Equations (2)–(4) implement temporal convolution with an exponentially decaying kernel. Here $\alpha_I = e^{-\frac{\Delta t}{\tau_I}}$, $\alpha_{AHP} = e^{-\frac{\Delta t}{\tau_{AHP}}}$, $\alpha_V = e^{-\frac{\Delta t}{\tau_V}}$, where τ_I , τ_{AHP} and τ_V are the decay constants of the corresponding exponentials. β is the update to the AHP current in response to an output spike. As the state transition was computed on Loihi, we associated a single compute step with 1 ms biological time, correspondingly $\Delta t = 1$ ms and $g_V = 1$.

Equation (2) defines the PSC as a function of input spikes arriving through incoming synapses of weights w_{ij} and delays of d_{ij} steps. The LIF neuron that is without AHP currents corresponds to the case of $\beta = 0$, where the neuron performs a leaky integration of $i_{PSC,j}[t]$ to get the membrane voltage $V_j[t]$. When this voltage exceeds a threshold, it is reset to zero and an output spike is generated. In this case, the memory of the neuron is limited by the voltage and PSC decay time constants τ_V and τ_I , respectively, which are typically around 20 ms. This means that even when connected in a recurrent fashion, the memory capacity for the network is typically at most 100 ms.

Equation (3) defines the AHP current. With $\beta > 0$, each output spike (that is, $z_j[t] > 0$) will cause $i_{AHP,j}[t]$ to become more negative by a value of β . When leaky integrated into the membrane voltage $V_j[t]$ (equation (4)), this increased negative value of $i_{AHP,j}[t]$ lowers the rate of subsequent spikes, leading to spike frequency adaptation. The decay time constant of the AHP current τ_{AHP} is much longer than τ_V , τ_I , typically > 100 ms. The slow decay means that this inhibition persists over a much longer duration, thus functioning as a longer-term memory cell. This longer-lasting memory proved invaluable to solve the complex tasks demonstrated in this work.

This model can be very efficiently implemented on Loihi, making use of its support for multi-compartment neuron models. The AHP current was calculated in the voltage register of a separate compartment. Loihi then allows this voltage to be integrated into the voltage of the main compartment as an additional current. This usage of multiple compartments to implement slow-changing dynamics differentiates our model from previous neuromorphic implementations of memory.

Details of spiking neural network training. In this section we describe important details pertaining to the training of networks of LIF neurons and AHP neurons. In all equations below, we drop the neuron index j for brevity.

The scaled voltage. For subsequent details regarding SNN training, we find it useful to define a normalized version of the membrane voltage; that is, a scaled voltage v_s .

We first noted that the membrane voltage $V[t]$ is a sum of two voltage components $V_{PSC}[t]$ and $V_{AHP}[t]$, which are a result of leaky integrating i_{PSC} and i_{AHP} respectively. Correspondingly, we could rewrite equations (4) and (6) describing the voltage evolution as follows:

$$V_{PSC}[t + \Delta t] = \begin{cases} \alpha_V V_{PSC}[t] + \frac{1}{g_V} i_{PSC}[t + \Delta t] & \text{if neuron is not refractory} \\ 0 & \text{otherwise} \end{cases}, \quad (7)$$

$$V_{AHP}[t + \Delta t] = \begin{cases} \alpha_V V_{AHP}[t] + \frac{1}{g_V} i_{AHP}[t + \Delta t] & \text{if neuron is not refractory} \\ 0 & \text{otherwise} \end{cases}, \quad (8)$$

$$V[t + \Delta t] = V_{PSC}[t + \Delta t] + V_{AHP}[t + \Delta t]. \quad (9)$$

$$\begin{aligned} V_{PSC,j}[t + \Delta t] &\rightarrow 0 \\ V_{AHP,j}[t + \Delta t] &\rightarrow 0 \end{aligned} \text{ if } z_j[t + \Delta t] = 1 \quad (10)$$

The scaled voltage $v_s[t]$ is defined below:

$$v_s[t] = \frac{V[t] - b_0}{b_0 - V_{AHP}[t]} \quad (11)$$

$v_s[t]$ takes a value of 0 when $V[t] = b_0$ and a value of -1 when $V[t] = V_{AHP}[t]$. This is motivated by the fact that $V_{AHP}[t]$ is the value that $V[t]$ would take if there was no input PSC.

The surrogate gradient. The generation of spikes from the membrane voltage (equation (5)), involves the use of a step function centred at the neuron threshold. This function is non-differentiable at the neuron threshold and provides a non-informative gradient of zero at all other points. Thus, to use gradient backpropagation to train networks of spiking neurons, we considered a surrogate gradient for the step function similar to methods used in previous works^{12,20–25}.

We rewrote the thresholding equation (5) in terms of $v_s[t]$:

$$z[t + \Delta t] = h(v_s) \equiv h\left(\frac{V[t] - b_0}{b_0 - V_{AHP}[t]}\right), \quad (12)$$

where h is the unit step function.

We then used the following piecewise linear surrogate gradient function to serve as a pseudoderivative of the step function $h(\cdot)$.

$$\frac{dh}{dv_s} \triangleq \begin{cases} \gamma \left(1 + \frac{v_s}{v_-}\right) & \text{if } -v_- \leq v_s < 0 \\ \gamma \left(1 - \frac{v_s}{v_+}\right) & \text{if } 0 \leq v_s \leq v_+ \\ 0 & \text{otherwise} \end{cases}, \quad (13)$$

where v_- and v_+ define the support of the surrogate gradient and γ is a dampening factor that affects the magnitude of the derivative.

Thus $h'(v_s)$ peaks at a value of γ for $v_s = 0$ and linearly decays to zero at the values of $-v_-$ and v_+ .

Impact of AHP neurons on gradient propagation. In this section we provide the theory that explains how gradients are protected and propagated further back in time in the presence of AHP neurons. In this case, we considered a gradient with a loss L that depends only on the output spikes $z[t]$.

For a standard LIF neuron, the following three equations govern gradient BPTT. For a neuron indexed i , the gradient L with respect to the state variables propagates as follows.

$$\begin{aligned} \frac{dL}{dV_i[t]} &= \alpha_V \frac{dL}{dV_i[t+\Delta t]} + \frac{\partial z_i[t]}{\partial V_i[t]} \frac{dL}{dz_i[t]} \\ &= \alpha_V \frac{dL}{dV_i[t+\Delta t]} + \frac{h'(v_{s,i}[t])}{b_0} \frac{dL}{dz_i[t]}, \end{aligned} \quad (14)$$

$$\begin{aligned} \frac{dL}{dI_{PSC,i}[t]} &= \alpha_I \frac{dL}{dI_{PSC,i}[t+\Delta t]} + \frac{\partial V_i[t]}{\partial I_{PSC,i}[t]} \frac{dL}{dV_i[t]} \\ &= \alpha_I \frac{dL}{dI_{PSC,i}[t+\Delta t]} + \frac{1}{g_V} \frac{dL}{dV_i[t]}, \end{aligned} \quad (15)$$

$$\frac{dL}{dz_i[t]} = \frac{\partial L}{\partial z_i[t]} + \sum_j w_{ij} \frac{dL}{dI_{PSC,j}[t + d_{ij} + \Delta t]}. \quad (16)$$

In equation (14), we see the gradient decay proportional to α_V through the membrane voltage. In equation (15), we see a decay proportional to α_I through the PSC. Both decay factors correspond to small decay time constants (5–20 ms) and thus do not preserve gradient information too far back. The gradient also propagates backwards from spikes to voltages to the PSC and then through the recurrent weights, by applying equations (14)–(16) in sequence. The decay of the gradient through this channel is proportional to $h'(v_{s,i}[t])$ and thus γ . γ is typically kept low because gradients that propagate via the pseudoderivative are approximations and become less informative the further they propagate. This also helps to keep the gradient propagation stable.

All of the above means that the informative gradients do not propagate too far back in time. However, with the addition of the slow-decaying AHP dynamics, we have the following:

$$\begin{aligned} \frac{dL}{dI_{AHP,i}[t]} &= \alpha_{AHP} \frac{dL}{dI_{AHP,i}[t+\Delta t]} + \frac{\partial V_i[t]}{\partial I_{AHP,i}[t]} \frac{dL}{dV_i[t]} \\ &= \alpha_{AHP} \frac{dL}{dI_{AHP,i}[t+\Delta t]} + \frac{1}{g_V} \frac{dL}{dV_i[t]}, \end{aligned} \quad (17)$$

$$\frac{dL}{dz_i[t]} = -\beta \frac{dL}{dI_{AHP,i}} + \frac{\partial L}{\partial z_i[t]} \sum_j w_{ij} \frac{dL}{dI_{PSC,j}[t + d_{ij} + \Delta t]}. \quad (18)$$

Equation (17) shows a means for gradients to propagate through the AHP currents with a very slow decay α_{AHP} , which allows the propagation of an informative gradient signal far back in time. This gradient signal is then propagated through the spikes (equation (18)) and consequently through the voltage and current and the weights.

Figure 2d demonstrates the impact that the AHP current has over the propagation of $\frac{dL}{dz_i[t]}$ over time. In the top plot, the slowly decaying gradient is visible in the $\left| \frac{dL}{dz_i[t]} \right|$ of the AHP neurons. This gradient then gives an informative gradient signal to the rest of network.

Spike rate regularization. For each neuron k , we calculated the mean rate $\bar{\rho}_k$ across all batches. We then added the following regularization loss

$$L_\rho = \lambda_\rho \left(\sum_j (\bar{\rho}_k - \rho_{\text{target}})^2 \right), \quad (19)$$

where ρ_{target} is a target rate and λ_ρ is the parameter that controls the strength of the regularization. This loss encourages the mean spike rate of each neuron across a random batch to be as close as possible to ρ_{target} . This ensures that the network activity does not die out and that the spike rate stays sparse owing to the low value

of ρ_{target} . The outermost square is included to dynamically reduce the regularization strength as the loss becomes smaller.

When training the spiking RelNet, we used a more aggressive spike rate regularization to limit the total spike rate across all the instances of g_ρ . This is described below in the section on the training of the spiking RelNet.

Details for the calculation of the PSPR. We considered two networks with randomly drawn weights, one with and one without AHP neurons. In each case we measured the extent by which images from two classes (handwritten 6 and 8) were separated by the spiking activity of the network after the input sequences were fed in. More precisely, we considered the 240-dimensional vector consisting of the low-pass-filtered spikes (filter time constant: 20 ms) from all network neurons at $T = 840$ ms. At that time point not only have the input sequences ended, but the spikes from a separate input neuron have also been provided from $T = 784$ ms to $T = 840$ ms. These serve as a ‘prompt’ that transforms values of hidden variables of the network states, the current values of AHP currents, into observable spiking activity. We normalized these vectors to 1, and then measured the Euclidean distance between any two such vectors that resulted from handwritten samples of the digits 6 and 8. We used 100 MNIST¹³ images from each of the two classes. Each image was presented as described in the section on Input encoding for the sMNIST task. The histogram of the resulting pairwise distances is plotted in Fig. 2c.

To mirror our sMNIST solution, we used a network of 240 LIF neurons and compared it with the AHP SNN with 140 LIF neurons and 100 AHP neurons.

To test how well the vectors that result from the two input digits 6 and 8 could be separated by a linear classifier, we trained a standard max-margin linear support vector classifier. We used 500 images per class.

Details of the application to sMNIST. Input encoding. The grey values of the pixels from an MNIST¹³ image were encoded in spikes using a threshold-crossing encoding (Fig. 2a). The number of thresholds was chosen to be 40, which is half of the 80 input neurons. So we had two neurons for each threshold, one spiking when the grey value between the former and the new pixel increased, and the other neuron spiking when the grey value decreased when transitioning to the new pixel. The thresholds were linearly spaced between 0 and 255. The pseudo code for the input encoding can be seen in Supplementary Section 1. After the presentation of all 784 pixels, which took 1 ms each, the 80th input neuron becomes active for an additional 56 ms. Thus, the presentation of one sample takes 840 ms. This last input neuron, which generates a spike at every time step for 56 steps after the image presentation, indicates the end of a sample. The classification happens at the last time step (that is, time step 840 of a sample). Each of the ten output neurons denotes a digit, and the neuron with the highest membrane potential on the last time step defines the predicted class. The network was implemented on the Intel Loihi chip using NxNet API from the NxSDK v0.95²⁶.

Network structure. An AHP SNN consisting of 240 neurons was used: 180 excitatory and 60 inhibitory. A random subset of 100 of the excitatory spiking neurons were AHP neurons; 80 of the input neurons were also used to perform an input spike encoding of the images, and 10 output neurons were used, corresponding to the 10 classes of the MNIST dataset. The hyperparameters that were used to train the network for Loihi were $\beta = 96$, baseline threshold $b_0 = 127$, $\tau_V = 20$ ms, $\tau_{AHP} = 700$ ms and a refractory period and delay of 1 ms.

Control models. An overview of the network structures from Bellec et al.¹² to which the AHP SNN was compared in Fig. 2e for solving the sMNIST task is given here. The SNN without AHP neurons consisted of 220 LIF neurons with 100% connectivity, the LSNN consisted of 120 LIF neurons and 100 neurons with adaptive thresholds with 12% connectivity, the recurrent ANN without LSTM units consisted of 128 hidden units with hyperbolic tangent activation function with 100% connectivity and the ANN with LSTM units consisted of 128 LSTM units with 100% connectivity.

Training details. The AHP SNN for solving sMNIST was trained for 465 epochs on the MNIST¹³ training set during two stages, similarly to the procedure used by Akopyan et al.⁸. The forward pass used a software model of Loihi neurons including the limited precision on neural states (for example, 8 bits for weights). For the backward pass the full precision weight matrix was kept to calculate the weight updates, which were quantized afterwards for the forward pass. During training, the overall connectivity of the network was kept at 20%, including the input and output connectivities. This meant that only 20% of the possible synapses between the neurons were active. This was achieved by using the rewiring technique DEEP-R¹⁴ during training, which is based on random synaptic sampling. Constraining the connectivity to 20% of the available connections was needed to stay within the limited axonal and synaptic resources of a Loihi neuro-core. The resulting sparsely connected AHP SNN had an average firing rate of 29 Hz per neuron on Loihi classifying the test set of MNIST¹³.

Spiking RelNet network architecture. In this section, we describe in detail the structure of the spiking RelNet as applied to the bAbI tasks.

Building on the general architecture proposed in ref.¹⁵, the spiking RelNet takes as input K objects $o_i(t)$, and a question object $q(t)$ and produces a network output according to equation (1).

Figure 3a shows the basic outline of this network. When applied to the bAbI task, the sentences of the story and the question were embedded into the spike sequences $o_i(t)$ and $q(t)$ respectively by means of AHP SNNs, which are recurrent networks consisting of LIF neurons and AHP neurons. To provide the input to the AHP SNN, we assigned an input neuron corresponding to each distinct word used in the bAbI dataset. The words in a sentence/question were then presented in sequence, with each word being presented for a duration of $T_{\text{word}} = 10$ ms during which only the corresponding input neuron fired continuously. We then took the spike activity of the AHP SNN over the last 14 ms, and padded it with a period of 23 ms without spikes to form the embedding spike sequences $o_i(t)$ and $q(t)$ (Fig. 3b).

The function g_o is the relational function. It receives as input a triplet of spike sequences ($o_i(t)$, $o_j(t)$, $q(t)$) corresponding to a pair of sentences and questions, and produces a spike sequence output. It is implemented as a four-layer feed-forward spiking neural network with LIF neurons. We have an instance of g_o for each pair of sentences i, j : $i \leq j$, so that the ordering of the sentences in the stories is made available to the network.

The function f_{agg} is an element-wise function. This is implemented by means of a LIF neuron layer to which each instance of g_o is connected one-to-one, where the set of input weights from an instance of g_o to this layer is shared across all instances. This is an addition to the architecture proposed in Santoro et al.¹⁵ and plays an essential role in enabling the implementation of spiking RelNets in neuromorphic hardware.

The function f_o is the readout function. It is implemented as a three-layer feed-forward LIF neuron network followed by a linear readout (see section below) and a softmax layer. f_o outputs, for each unique word present in the bAbI dataset, the probability of that word being the answer to the question. This probability is used to compute a cross-entropy loss that is used to train the network using gradient backpropagation. For more detailed parameters pertaining to the layers, see Supplementary Table 5.

Optimizing the performance and energy efficiency of the spiking RelNet.

The network outline above shows that the spiking RelNet uses both recurrent networks for embedding (b in Fig. 3a) as well as feed-forward networks for g_o and readout (c–e in Fig. 3a) to perform the calculation. When scaling up to tasks with a larger number K of sentences, the fraction of these feed-forward components of the spiking RelNet increases (Fig. 4b). The reason is that the number of sentences to be embedded, and hence AHP SNN Networks, scales linearly with K , whereas the number of feed-forward modules that compute the function g_o increase quadratically with K , since we have an instance of g_o for each pair of sentences. Consequently, the numerous instances of g_o occupy the majority of the hardware resources (Fig. 5c). This increasing fraction of feed-forward network modules is problematic from the perspective of energy efficiency, as previous emulations of feed-forward networks in spike-based hardware demonstrated that their advantage regarding energy consumption gets lost for larger networks when high classification accuracy is required¹. The reason is that these previous implementations had to use spike rate coding instead of event-based processing to achieve high classification accuracy. However, rate coding uses many spikes per neuron, thereby moving the network out of an energy-efficient working regime, and also increases the computation time of the network (that is, reduces its throughput).

The spiking RelNet also requires more computing time (time steps) than the non-spiking RelNet, making the loss computation and gradient BPTT many times more expensive in the spiking case. Simply training the network end-to-end with the cross-entropy loss requires an impractically long time for the network to converge, and also leads to pathological spike rates and low performance. The solutions to these issues are as follows.

Pre-training the AHP SNN. To speed up the convergence of the training algorithm, we first trained a non-spiking relational network to solve the bAbI tasks, where LSTMs were used to embed the questions and words. We then trained the AHP SNN to reproduce the outputs of the LSTMs for the various input sentences in the dataset. The weights of these pre-trained AHP SNNs were fixed, and they were used to perform the embedding while we trained g_o and f_o .

The AHP-SNN for sentences were pre-trained for 3,500 epochs with 5,218 data points per epoch. For the questions 38,000 epochs with 478 points per epoch were used. Using the pre-trained LSNN, training for the spiking RelNet in contrast took only 80 epochs, with 17,000 data points per epoch, demonstrating the faster convergence achieved due to pre-training. Despite the larger number of epochs, pre-training the embedding was cheaper (15h using a single GPU) owing to the much larger size of the spiking RelNet (which took over a week on two GPUs for 80 epochs). The resultant fewer training epochs needed compared with the end-to-end-trained non-spiking RelNet makes the training feasible for a spiking RelNet.

Compressing the network in time. Another aspect of optimization concerns the number of time steps used (which represents the computing time), which affects

not only the energy consumed and delay on Loihi, but also the training speed. We found that using spiking neurons without a refractory period and τ_{AHP} of just 7 ms reduced the required number of time steps considerably, while causing only a mild decrease in accuracy. In Supplementary Table 3 we show that a network whose dynamics are stretched over more time steps can solve all 17 tasks at the cost of higher energy consumption and latency.

Aggressive spike rate regularization and voltage regularization. The energy inefficiency of DNNs owing to higher spike rates, which is common with CNNs, can be overcome in the case of the spiking RelNet. An important underlying difference from CNNs is that even for large problem instances (that is, stories with many sentences), the number of relations that are relevant to answering a question tends to increase only linearly with the length of the story.

We used a more aggressive regularization for the spike rates in the instances of g_o , where the regularization forces the total spike rate summed over all instances of g_o towards a low target rate. This forced the network to allocate spiking activity only for those instances of g_o that corresponded to relevant pairs of sentences. The resultant low spike rate seen in Fig. 4b resulted in a very power-efficient and delay-efficient implementation of feed-forward spiking networks in neuromorphic hardware. See Supplementary Section 4.4 for the precise formulation and more details.

Voltage regularization. The strong spike rate regularization described above has a tendency to push the synaptic weights low enough that the membrane voltages become very negative. This leads to a large number of time steps where the voltage values fall outside the support of the surrogate gradient and thus no gradient information can be propagated through them, which impedes gradient backpropagation. Thus we were motivated to add a loss that penalizes voltages that fall sufficiently outside the support of the surrogate gradient function defined in equation (13). As the surrogate gradient is defined in terms of v_s (equation (11)), we define the voltage regularization loss in terms of it as well.

For each neuron j and time step n , we calculated the loss component

$$L_v^{(jn)} = (\text{relu}(v_{s,j}[n] - 0.4))^2 + (\text{relu}(-v_{s,j}[n] - 2.0))^2. \quad (20)$$

The total voltage regularization loss is given by

$$L_v = \lambda_v \left(\text{mean}_{i,n} L_v^{(i,n)} \right)^2. \quad (21)$$

This loss was chosen with the support of the surrogate gradient $v_s \in [v_-, v_+] = [-1.0, 0.5]$ in mind. Hence the above penalizes all neurons at all time instants where v_s goes outside the range $[-2.0, 0.4]$. This prevented the network from using voltages that are excessively negative and increases the proportion of voltage values that lie within the support of the surrogate gradient. Moreover, limiting the range of the voltage values was also crucial to be able to fit the voltage values onto the range offered by the fixed precision registers on Loihi.

The linear readout. The design of the linear readout was crucial to the performance of the relational network. The linear readout consists of a network of specialized readout neurons, with one neuron for each word in the database of words used in the bAbI task.

The readout neuron is a variant of the standard LIF neuron, where $\tau_i = \tau_{\text{readout}} = 7.0$ ms and $\tau_v = \infty$, and the threshold $b_0 = \infty$. This corresponds to a neuron that does not spike, but where the PSC decays with constant τ_{readout} and the neuron performs (non-leaky) integration of the PSC to calculate the membrane potential. However, we chose to enable the integration of the PSC into the voltage only $T_{\text{readout}} = 10$ ms before the final step. The value of the membrane voltage at the final step was scaled by a fixed scalar and formed the input to the softmax (Fig. 3e). This design incentivized the spike activity of the final layers to occur in a confined time window close to the final time step, while allowing the precise timing of the spike to influence the final output, leading to a high information capacity in a short time window.

Placement of the spiking RelNet on Loihi. The Loihi Nahuku board consists of 32 interconnected Loihi chips, each of which contains 128 neuro-cores. The neuro-core is the fundamental computational unit that computes the dynamics of the LIF neurons and AHP neurons. Loihi allows one to connect any neuro-core on any chip to any other neuro-core on any other chip, thus enabling large networks to be placed on the board. However due to hardware limitations, the number of connections and the connectivity is constrained as described below. Transporting a large number of spikes across different chips also incurs considerable latency. We discuss here the strategies to place the spiking RelNet within these constraints.

The AHP SNN network that solves the sMNIST task contains 240 neurons connected with 20% of the recurrent connections enabled. This network is small enough to fit in a single chip and occupy only one neuro-core. The spiking RelNet is a much larger network. Considering a maximum of $M = 20$ sentences in a story, the spiking RelNet has M instances of the AHP SNNs that embed sentences, plus one for the questions. There also exists an instance of g_o for each pair of sentences

$o_i(t), o_j(t): i \leq j$, making a total of $\frac{M(M+1)}{2} = 210$ instances. Each of these instances is implemented as a separate network on Loihi, leading to a total network size of 238,604 neurons. The placement of this network needs to take into consideration many constraints regarding connectivity, memory and the latency of spike transport. The associated challenges and solutions are outlined in this section.

Synaptic memory limit. Each neuro-core has a limited amount of SRAM memory that can be used to store synaptic parameters. This limits the number of incoming synapses to a particular neuro-core. The precise number is dependent on synaptic parameters and we have found an empirical limit of around 40,000 synapses per neuro-core. Apart from the aggregation layer, all layers in the network have dense input and recurrent synaptic connections. Thus each layer needs to be placed over multiple neuro-cores to store the input and recurrent connections.

Fan-out limits. The total number of neuro-cores to which the neurons of a neuro-core connect to is limited to 2,048, and 4,096 for intra-chip connections. This plays a role when connecting the AHP SNNs to the large number of instances of g_o .

Thus, one can split the neurons across multiple neuro-cores to reduce the number of output connections per neuro-core. However, splitting a recurrent AHP SNN network across too many neuro-cores increases latency. Instead we used relay layers. A relay layer, as the name suggests, simply reproduces the spiking activity of the layer that forms its input. Each AHP SNN was thus connected to multiple relays, which then each fan out to a smaller number of instances of g_o .

Limits pertaining to fan-in. For any neuro-core C , Loihi limits the number of neurons that can be connected to that neuro-core to 4,096. Unlike the two constraints above, this constraint on the fan-in to a neuro-core introduces a fundamental restriction to the network architectures that can be implemented on Loihi.

The layer that receives the output from the instances of g_o receives input from $\frac{M(M+1)}{2} = 210$ instances. To avoid violating the fan-in constraint, the connection from the output of g_o to this layer must be sparse. Thus, we introduced an aggregation layer to which each instance of g_o was connected in a sparse one-to-one manner, with shared weights across instances. The sparse connection enabled the aggregation layer to be implemented within the fan-in constraints.

For a more detailed treatment of the constraints, as well as the number of neuro-cores required to place each layer, see Supplementary Sections 4.5 and 4.6.

Reducing cross-chip spike transport. When placing the AHP SNN, relay and the relational networks taking into consideration only the connectivity constraints, we noticed large delays that occurred due to transporting spikes from the AHP SNN and relay networks to the instances of g_o . This is owing to the large number of spikes that need to be transferred across different Loihi chips. Thus, in addition, we needed to optimize the placement of the instances of g_o and the relay networks in a manner that minimized cross-chip spike transport. We broke down this general objective into the following constraints.

- All relay networks must be connected only to g_o instances that are placed on the same Loihi chip. We thus chose to place the initial layer of the g_o instances in the same chip as the relay networks that give them their input.
- We aimed to minimize the number of relay networks required. Each chip had a limit of 128 neuro-cores and thus a limit on the number of g_o instances that can be placed. This means that for each chip, we must choose the set of g_o instances in such a manner that the number of distinct sentences needed as input is minimized.
- For each g_o instance, all layers after the first are to be placed on the same chip.
- The layout that we arrived at with the above principles is described in the Supplementary Section 4.6. The resultant improvement in delay and the corresponding EDP is shown in Fig. 5e.

Data availability

The MNIST dataset¹³ is freely available at <http://yann.lecun.com/exdb/mnist>. The bAbI dataset¹⁶ is freely available at <https://research.fb.com/downloads/babi>.

Code availability

The Loihi source code is freely available from Github at https://github.com/intel-nrc-ecosystem/models/tree/master/nxsdk_modules_ncl/lenn.

Received: 30 June 2021; Accepted: 20 March 2022;
Published online: 19 May 2022

References

1. Davies, M. et al. Advancing neuromorphic computing with Loihi: a survey of results and outlook. *Proc. IEEE* **109**, 911–934 (2021).
2. Benda, J. & Herz, A. V. M. A universal model for spike-frequency adaptation. *Neur. Comput.* **15**, 2523–2564 (2003).

3. Gutkin, B. & Zeldenrust, F. Spike frequency adaptation. *Scholarpedia* **9**, 30643, revision 14332 <https://doi.org/10.4249/scholarpedia.30643> (2014).
4. Allen Institute Brain Atlas: Cell Feature Search (Allen Institute, accessed 3 August 2021); <https://celltypes.brain-map.org/data>
5. Davies, M. et al. Loihi: a neuromorphic manycore processor with on-chip learning. *IEEE Micro* **38**, 82–99 (2018).
6. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neur. Comput.* **9**, 1735–1780 (1997).
7. Shrestha, A. et al. A spike-based long short-term memory on a neurosynaptic processor. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* 631–637 (IEEE, 2017).
8. Akopyan, F. et al. Truenorth: design and tool flow of a 65 mW 1 million neuron programmable neurosynaptic chip. *IEEE Trans. Comput. Aid. Des. Integr. Circ. Syst.* **34**, 1537–1557 (2015).
9. Lotfi Rezaabad, A. & Vishwanath, S. Long short-term memory spiking networks and their applications. In *International Conference on Neuromorphic Systems 2020* 3 (Association for Computing Machinery, 2020); <https://doi.org/10.1145/3407197.3407211>
10. Maass, W., Natschläger, T. & Markram, H. Real-time computing without stable states: a new framework for neural computation based on perturbations. *Neur. Comput.* **14**, 2531–2560 (2002).
11. Furber, S. B., Galluppi, F., Temple, S. & Plana, L. A. The SpiNNaker project. *Proc. IEEE* **102**, 652–665 (2014).
12. Bellec, G., Salaj, D., Subramoney, A., Legenstein, R. & Maass, W. Long short-term memory and learning-to-learn in networks of spiking neurons. In *Advances in Neural Information Processing Systems* Vol. 31 (eds Bengio, S. et al.) 795–805 (Curran Associates, Inc., 2018).
13. LeCun, Y., Cortes, C. & Burges, C. *MNIST Database of Handwritten Digits* (ATT Labs, 2010); <http://yann.lecun.com/exdb/mnist>
14. Bellec, G., Kappel, D., Maass, W. & Legenstein, R. Deep rewiring: training very sparse deep networks. In *International Conference on Learning Representations* (2018).
15. Santoro, A. et al. A simple neural network module for relational reasoning. In *Advances in Neural Information Processing Systems* (Ed. Guyon, I., Von Luxburg, U., et al.) **30**, 4967–4976 (NIPS, 2017).
16. Weston, J. et al. Towards AI-complete question answering: a set of prerequisite toy tasks. Preprint at <https://arxiv.org/abs/1502.05698> (2015).
17. Bellec, G. et al. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nat. Commun.* **11**, 3625 (2020).
18. Scherr, F., Stöckl, C. & Maass, W. One-shot learning with spiking neural networks. Preprint at *bioRxiv* <https://doi.org/10.1101/2020.06.17.156513> (2020).
19. Billeh, Y. N. et al. Systematic integration of structural and functional data into multi-scale models of mouse primary visual cortex. *Neuron* **106**, 388–403 (2020).
20. Zenke, F. & Vogels, T. P. The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neur. Comput.* **33**, 899–925 (2021).
21. Esser, S. K. et al. Convolutional networks for fast, energy-efficient neuromorphic computing. *Proc. Natl. Acad. Sci. USA* **113**, 11441–11446 (2016).
22. Shrestha, S. B. & Orchard, G. Slayer: Spike layer error reassignment in time. In *Advances in Neural Information Processing Systems* Vol. 31 (eds Bengio, S. et al.) (Curran Associates, Inc., 2018).
23. Neftci, E. O., Mostafa, H. & Zenke, F. Surrogate gradient learning in spiking neural networks: bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Process. Mag.* **36**, 51–63 (2019).
24. Zenke, F. & Ganguli, S. SuperSpike: supervised learning in multilayer spiking neural networks. *Neur. Comput.* **30**, 1514–1541 (2018).
25. Zhu, X., Zhao, B., Ma, D. & Tang, H. An efficient learning algorithm for direct training deep spiking neural networks. *IEEE Trans. Cogn. Dev. Syst.* (2021).
26. Florey, D. (2020, December 9). Neuromorphic Software Overview. Neuromorphic Software Overview - INRC Public - Confluence. Retrieved April 26, 2022, from <https://intel-ncl.atlassian.net/wiki/spaces/INRC/pages/524354/Neuromorphic+Software+Overview>

Acknowledgements

This research/project was supported by the Human Brain Project (grant agreement number 785907 and 945539, both to W.M.) of the European Union and a grant from Intel (to W.M.). Special thanks go to G. Bellec and D. Salaj for their insightful comments and ideas when carrying out this work.

Author contributions

A.R., P.P. and W.M. contributed to the design and planning of the experiments. A.R. and P.P. carried out the experiments. A.R., P.P., A.W. and W.M. participated in the analysis of the experimental data. A.R., P.P., A.W. and W.M. wrote the manuscript.

Competing interests

P.P. and A.W. are currently employed by Intel Labs, developers of the Loihi neuromorphic system. W.M. and A.R. are members of the Intel Neuromorphic Research Community and W.M. has received research funding from Intel for related work.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s42256-022-00480-w>.

Correspondence and requests for materials should be addressed to Wolfgang Maass.

Peer review information *Nature Machine Intelligence* thanks Jianhua Yang, Tara Hamilton and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

© The Author(s), under exclusive licence to Springer Nature Limited 2022